Kelvin

C を、少しずつ変えていく。

C の静かな代替。ABI は 100% 互換のまま。

Kelvin は綴りの違う C

`kelvinc` は Kelvin を C に翻訳し、あなたの C コンパイラに渡す。



型もレイアウトも呼び出し規約も意味論も、作りからして C のもの。

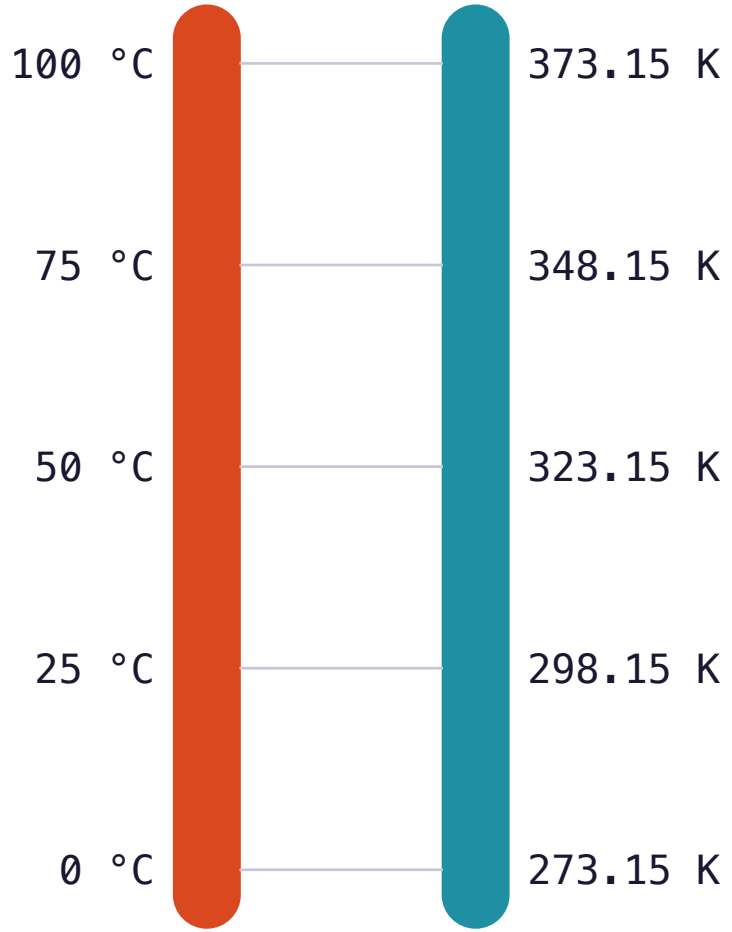
名前が設計そのもの

Dan
kelvin の実装を始めよう。`better C` を実装する試みだ。名前は事実上あなたがつけた:

Claude
私なら **Kelvin** と呼びます。

C は摂氏 (Celsius) の記号でもあり、Kelvin はその自然な後継です。目盛りの幅は同じなので、C について知っていることはすべて一対一で持ち越せます…

Kelvin に決めたのは、この駄洒落がそのまま設計仕様でもあるからです。



Dialogue.md 第1節・第2節の和訳。次点は Sequel、P、Coda、Seam。

宣言は左から右へ読む

C

```c
#include <stdint.h>

struct point { int64_t x, y; };

int64_t add(int64_t a, int64_t b) {
    return a + b;
}

int main(void) {
    const int64_t k = 7;
    int32_t n = 0;
    int32_t *p = &n;
    *p = 9;
    return n;
}
```

Kelvin

```
struct point { x: i64; y: i64 }

let add(a:i64, b:i64):i64 {
    return a + b
}

let main():i32 {
    let k = 7
    var n:i32 = 0
    var p:i32^ := &n
    p^ = 9
    return n
}
```

C は C のまま

fromc.k

```
#import <stdio.h> as C     // C のヘッダそのもの

let square(n:i32):i32 { return n * n }

let main():i32 {
    let k = 7              // i64 の let
    var total:i32 = 0
    var p:i32^ := &total   // C の int32_t *
    p^ = square(3)
    printf("%d %lld\n", total, k)
    return 0
}
```

ヘッダはそのまま読み込む

優先順位と型変換は C のもの

未定義動作も、書いたとおりに残る

出力

```
9 7
```

書き出される C が読める

sum.k

```
struct point { x: i64; y: i64 }

let sum(p:point):i64 {
    return p.x + p.y
}

let main():i32 {
    var total:i64 = 0
    for i in 0..<3 { total += i }
    return i32(total)
}
```

kelvinc --emit-c sum.k

```
struct point
{
    int64_t x;
    int64_t y;
};
int64_t sum(const struct point p)
{
    return p.x + p.y;
}
```

let の引数はただの const。Kelvin の struct は C の struct で、Kelvin の関数は C の関数。

なぜ C なのか、2026 年に

C は自分の足を撃てる言語で、米国政府も避けよと言っている。

反対意見。どれも正しい

2024
ホワイトハウスがメモリ安全な言語を求め、C と C++ を名指しした

2025
NSA と CISA も同じことを説く

いま
Go、Rust、Swift はメモリ安全で、ネイティブバイナリも出せる

それでも Kelvin は C の上に立つ

1. **libc** (macOS なら libSystem) はなくならない
2. **C コンパイラ**は最も基本のコンパイラ基盤で、思いつくどのプラットフォームにもある
3. **C の上に立つ**ので、大きさは 10 分の 1 以下
4. **メモリ安全性**は C の機能の範囲で実現でき、見た目は Rust や Swift に近づけられる

libc と cc はもうそこにある

Mac でビルドした hello world の otool -L

```
Kelvin  /usr/lib/libSystem.B.dylib

Rust    /usr/lib/libSystem.B.dylib

Go      /usr/lib/libSystem.B.dylib
        /usr/lib/libresolv.9.dylib

Swift   /usr/lib/libSystem.B.dylib
        /usr/lib/swift/libswiftCore.dylib
```

Go 1.12 のリリースノート: "libSystem is now used when making syscalls on Darwin." (Darwin のシステムコールは libSystem 経由になった)

libc は残る: ここでは 4 つとも、メモリ安全な 3 つも含めてリンクしている

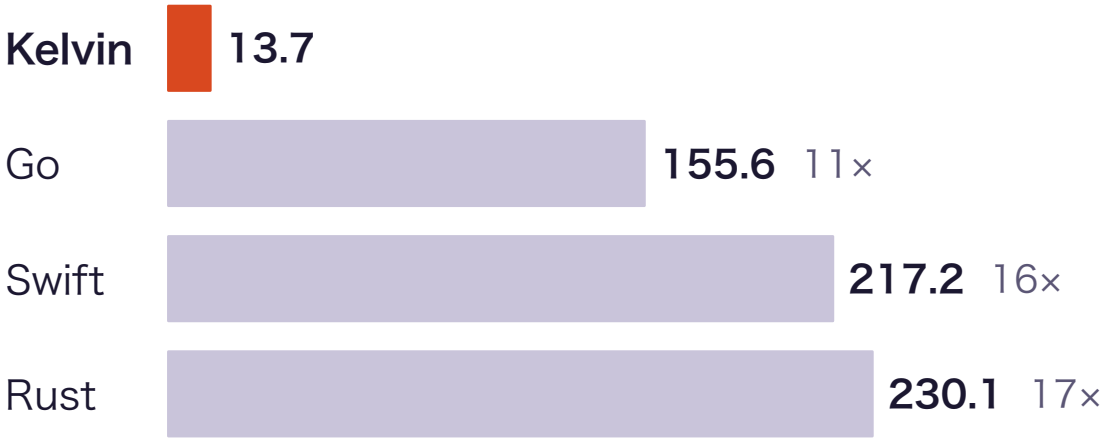
cc はどこにでもある: 最も基本のコンパイラ基盤で、思いつくどのプラットフォームにもある

だから Kelvin は両方を再利用し、移植の必要がない

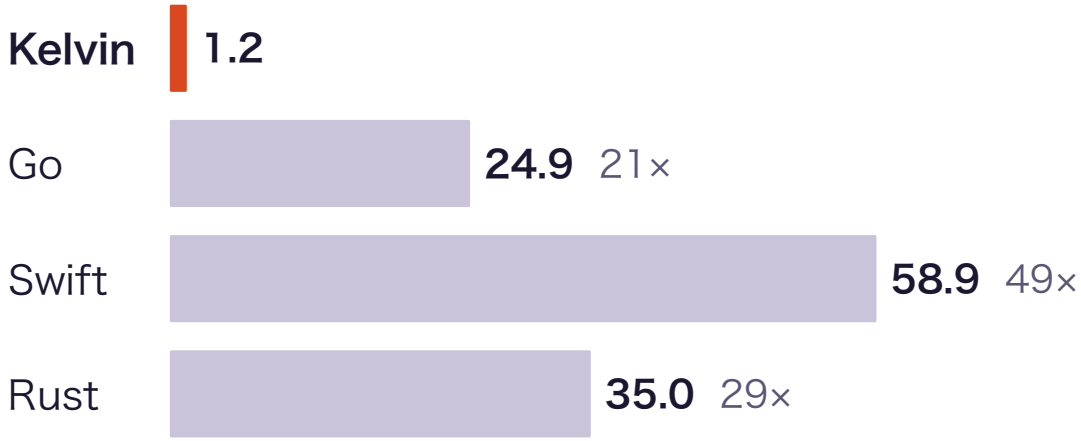
C の上に立つから Kelvin は小さい

10 分の 1 以下の大きさで、しかもその半分以上は Unicode データ。

追跡している全ファイル (MB)



コンパイラ自身のソース (MB)



Kelvin の 13.7 MB の内訳



2026-11-07 に `intro/measure.sh` で各既定ブランチを計測。LLVM と C コンパイラはどれにも数えていない。他の 3 つはずっと多くのことをこなしている。

見た目は Rust や Swift、変換先は C

safer.k

```
let half(n:i64):i64? {
    if n % 2 != 0 { return none }
    return n / 2
}

let main():i32 {
    let s = $"owned"
    let h = half(7)
    println(s, " ", h)
    return 0
}
```

kelvinc --emit-c safer.k

```
struct _kv_opt_i64 half(const int64_t n)
{
    if (n % 2 != 0)
    {
        return _kv__kv_opt_i64_make_none();
    }
    return _kv__kv_opt_i64_make_some(n / 2);
}

__attribute__((cleanup(kv_string_free)))
kv_string s = KV_STRING_LIT("owned");
const struct _kv_opt_i64 h = half(7);
```

所有者は自動で解放され、`T?` は取り出さないと使えず、`xs[i]` は検査される。すべて素の C で。`^` ポインタと C の未定義動作は残るので、Kelvin は「より安全な C」であり、Rust の保証はない。

$ ひとつで、二つの世界が共存する

C 本来のリテラルはそのまま。$ を一文字つければ、柔軟な版が隣に並ぶ。

C 本来: 硬直、メモリ安全でない		$ つき: 柔軟、メモリ安全	
`"text"`	cstr: 固定バイト列への生ポインタ	`$"text"`	String: バイト列を所有し、伸びる
`[1, 2, 3]`	固定配列: a[7] を誰も検査しない	`$[1, 2, 3]`	Array: 伸びる。xs[7] はプログラムを止める
`{1, 2}`	struct のフィールドを順に	`${"ann": 31}`	Dictionary。${1, 2} は Set
`// text`	コメント: C にこのリテラルはない	`$/k.l/i`	RegExp: パターンはコンパイル時に検査

:= で中へのポインタを取らないかぎり、$ の側は C のようには誤用できない。

「人間としては、短縮形がほしくなる」 Dan、$ が生まれた日に

ポインタは少し面倒にした。わざと

C: かつて称賛された定石

```c
#include <stdio.h>

int main(void) {
    char a[4] = "abc", b[4];
    char *p = b;
    const char *q = a;
    while ((*p++ = *q++) != '\0')
        ;
    puts(b);
    return 0;
}
```

Kelvin: 一行に一手

```
let main():i32 {
    var a:[u8](4) = "abc"
    var b:[u8](4)
    var p:u8^ := &b[0]      // ポインタは :=
    var q:u8^ := &a[0]
    while q^ != 0 {
        p^ = q^
        p++                 // これは文
        q++
    }
    p^ = 0
    println(&b[0] as cstr)
    return 0
}
```

```
while (p++^ = q++^) != 0 { }
```

```
error: '++' is a statement in Kelvin, not a value: step the pointer on a line of its own
```

```
p = q
```

```
error: 'p' is a reference (i32^): assign it with ':=', not '='
```

所有者は自分で解放される

owners.k

```
let shout(name:cstr):String {
    var s = String(name)    // バイト列を所有
    s += "!"
    return s                // 呼び出し元へムーブ
}

let main():i32 {
    let hi = shout("hello")
    var xs = $[3, 1, 2]     // Array<i64>
    xs += 4
    println(hi, " ", xs, " ", xs.count)
    return 0
}                           // hi と xs はここで解放
```

ブロックは変数が所有するものを解放する

return と引数渡しは所有者をムーブする

= はコピーしない: `.copy()` と書くか、`&` で借りる

出力

```
hello! [3, 1, 2, 4] 4
```

間違いは Kelvin の言葉で告げられる



```
int x = 3
```

```
error: 'int' is not a Kelvin type; use i32 or u32
```

```
let n = 0
n += 1
```

```
error: 'n' is a let and cannot change; declare it with var
```

```
let a = $"x"
take(a)
println(a)
```

```
error: 'a' was moved out and holds nothing now: assign it
anew before using it
```

```
let m:i32? = 3
println(m + 1)
```

```
error: i32? has no operator '+': compare with == and !=
```

失敗は値である

result.k

```
#import <stdio.h> as C
#import <errno.h> as C

let open(path:cstr):Result<FILE^, i32> {
    let f:FILE^ := fopen(path, "r")
    if f.isNull { return err(errno) }
    return f                          // T は ok(T) になる
}

let check(path:cstr):Result<i64, i32> {
    let f := try open(path)           // err は上へ
    fclose(f)
    return 1
}

let main():i32 {
    println(check("/no/such/file"))
    let first:i64? = none
    println(first ?? 42)
    return 0
}
```

T? と **Result<T, E>** の中身は C の struct

try はエラーを上へ渡す

?? は代わりの値を与える

出力

```
err(2)
42
```

どの型にもメソッドと演算子を足せる

money.k

```
struct money { cents: i64 }

extend money {
    static let +(a:Self, b:Self):Self { (Self){a.cents + b.cents} }
    static let <(a:Self, b:Self):bool { a.cents < b.cents }
    let dollars:f64 { .cents / 100.0 }
    let doubled():Self { self + self }
}

let main():i32 {
    let a:money = {150}
    let b:money = {275}
    var bills = $[b, a]
    bills.sort()                // 自前の < で
    println(bills)
    println((a + b).cents, " ", a.doubled().dollars)
    return 0
}
```

メソッドと属性は `extend T { }` に

演算子は型で選ばれる関数

C から見ればただの関数

出力

```
[{cents: 150}, {cents: 275}]
425 3.0
```

コレクションとテキストは組み込み

values.k

```
struct point { x: i64; y: i64 }

let main():i32 {
    let squares = (1...5).map { $0 * $0 }
    let even = squares.filter { $0 % 2 == 0 }
    let ages = ${"ann": 31, "bob": 42}
    let name = $"ann"
    println(squares, " ", even)
    println(squares.sorted { $0 > $1 })
    println(ages)
    println((point){3, 4})
    println(`${name} is ${ages["ann"]!}`)
    return 0
}
```

`$[...]` と **`${...}`** で Array、Dictionary、Set を作る

`{ $0 * $0 }` は無名関数で、クロージャではない

`println` はどんな値でも表示する

出力

```
[1, 4, 9, 16, 25] [4, 16]
[25, 16, 9, 4, 1]
{"ann": 31, "bob": 42}
{x: 3, y: 4}
ann is 31
```

async と await の実体はスレッド

tasks.k

```
let count(lo:i64, hi:i64):i64 {
    var n:i64 = 0
    for i in lo..<hi { n += i % 3 == 0 ? 1 : 0 }
    return n
}
let label(n:i64):String { $`${n} of them` }

let main():i32 {
    var parts:$[Task<i64>] = $[]
    for q in 0..<4 {
        parts += async count(q * 1000, (q + 1) * 1000)
    }
    var total:i64 = 0
    for i in 0..<parts.count { total += await parts[i] }
    let text = await async label(total)
    println(text)
    return 0
}
```

async f(x) は f を専用のスレッドで走らせる

await は待って、その値を返す

所有者は呼び出し元へムーブで戻る

出力

```
1334 of them
```

C からも Kelvin を呼べる

geo.k

```
struct point { x: f64; y: f64 }

let dist2(a:point, b:point):f64 {
    let dx:f64 = a.x - b.x
    let dy:f64 = a.y - b.y
    return dx * dx + dy * dy
}

// kelvinc --lib geo.k で
// geo.kh、libkgeo.a と
// libkgeo.dylib か libkgeo.so ができる
```

main.c

```
#include <stdio.h>
struct point { double x, y; };
double dist2__point__point(struct point a,
                           struct point b);

int main(void) {
    struct point a = {0, 0}, b = {3, 4};
    printf("%.1f\n",
           dist2__point__point(a, b));
    return 0;
}
```

Kelvin からは `#import <geo>`。C からはふつうのライブラリ。

出力

```
25.0
```

ライブラリは一つずつ import する

IO
ファイル、パイプ、プロセス

OS and SYS
ディレクトリ、環境変数、プロセス

JSON
`JSON.parse` と `JSON.stringify` を JS のとおりに

SION
SION の値。JSON の拡張

MsgPack
SION の MessagePack

RegExp
`$/pattern/flags`。JS の正規表現

UnicodeData
正規化、書記素、大文字小文字の変換

math
float と複素数のための `math.h`

自作
`kelvinc --lib M.k` で作れる

どれも `#import <M>` で読み込む。import しないものは入らない。

合意した変更を一つずつ

472

回の番号つき変更

1,205

個のテスト。clang と gcc 15 で

9

個の CI ジョブ。macOS、Ubuntu、FreeBSD

「ちょっと、先走りすぎだよ!」

初日のコンパイラは、トラップもスライスも `defer` も一度に足した。それを捨てて素の C に戻り、以来 C から離れるたびに一つずつ提案し、合意してきた。

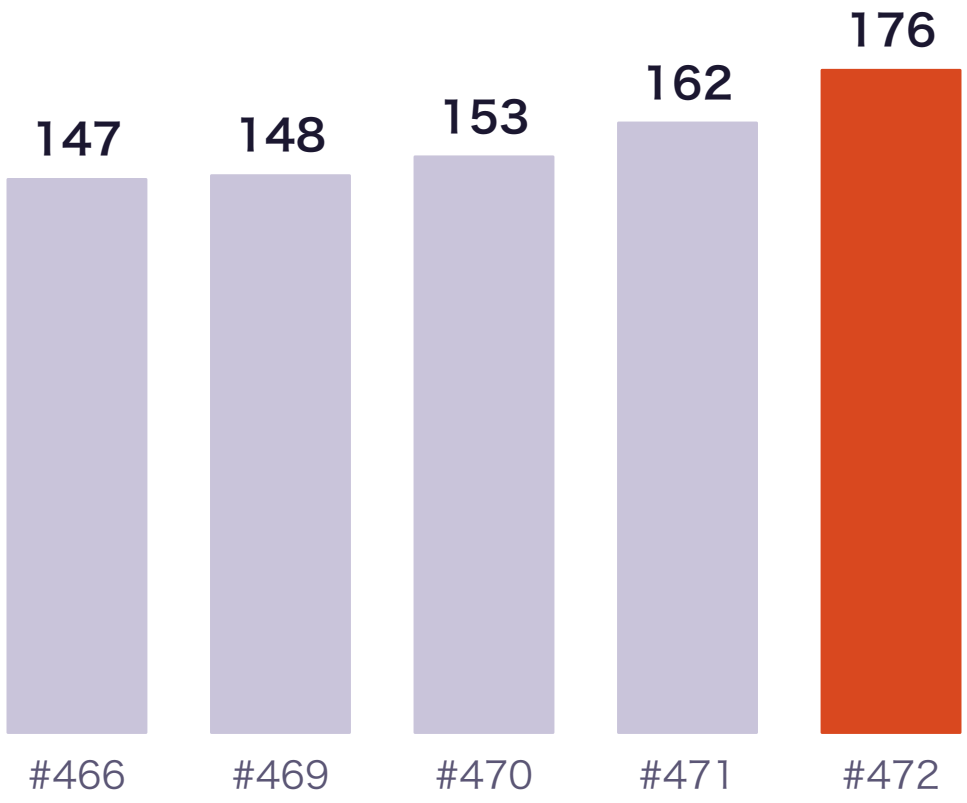
Design.md は何を決めたかを記録する

Dialogue.md はその経緯を、交わした言葉のまま記録する

Kelvin は自分のコンパイルを学習中

kc は Kelvin で書かれたコンパイラ。自分自身を不動点までビルドする。push のたびに CI で。

kc がビルドできるプログラム数 (番号つき変更ごと)



いま	テストプログラムと例の 354 本中 176 本
次に	`kelvinc` と同じ能力へ
その先	C を飛ばし、kc からアセンブリか LLVM IR へ
いつか	バックエンドもすべて Kelvin で

試してみる

```
git clone https://github.com/dankogai/kelvin
cd kelvin
make
./kelvinc --run examples/hello.k
./kelvinc --emit-c examples/hello.k
make install PREFIX=~/.local
```

必要なもの: C11 コンパイラ、GNU make、Python 3

動く環境: macOS、Linux、FreeBSD。Windows は不可

状態: 0.1.0。今後の 0.x で変わることがある

最初に読む: `documents/versusC.md`

VS Code なら: `make vscode`

ライセンス: MIT

C の知識は、すべて持ち越せる

github.com/dankogai/kelvin

MIT ライセンス